

This call allows the current scene to send information to all the other scenes. The `sendID` uniquely identifies the information you are sending. The `info` parameter is the information you want to send to all scenes.

A typical call would look like this:

```
self.sendAll(SOME_INFO_CONSTANT, data)
```

This effectively says, “Send information to all scenes. The information is identified by `SOME_INFO_CONSTANT`, and the information is in the value of the variable `data`.”

For this to work, all scenes other than the current scene must implement the `receive()` method, as described in the previous section. The scene manager sends the message to all scenes (other than the current scene). The current scene may contain a `receive()` method for information sent by other scenes.

Testing Communications Among Scenes

The Scene Demo program (with Scene A, Scene B, and Scene C), discussed earlier with Listings 15-2 and 15-3, contains code in each scene that demonstrates calls to `send()`, `request()`, and `sendAll()`. In addition, the scenes each implement simple versions of the `receive()` and `respond()` methods. In the demo program, you can send a message to another scene by pressing A, B, or C. Pressing X sends a message to all scenes. Pressing 1, 2, or 3 sends a request to get data from a target scene. The target scene responds with a string.

Implementation of the Scene Manager

Here we’ll look at how the scene manager is implemented. However, one important lesson of OOP is that the developer of client code doesn’t need to understand the implementation of a class, only the interface. With respect to the scene manager, you don’t need to know how it works, only what methods you must implement in your scenes, when they are called, and what methods you can call. Therefore, if you’re not interested in the internals, you can go right to the Summary. If you are interested, this